

Lista de Código: CRUD em Arrays

Turma: LionsDev

Tópicos: array de objetos, `.push()` (Create), `.find()` e `.filter()` (Read), `.findIndex()` (Update) e `.filter()` (Delete), o CRUD completo em memória.

Nesta lista você implementa as funções de um CRUD sobre um array de objetos. Comece sempre com `let itens = []` e um contador de id. Cada exercício descreve o comportamento esperado.

Parte 0 — Treino rápido (aquecimento)

Considere `const produtos = [{ id: 1, nome: "Mouse" }, { id: 2, nome: "Teclado" }]`.

1. Adicione `{ id: 3, nome: "Monitor" }` ao array com `.push()`.
2. Imprima quantos produtos existem (`.length`).
3. Use `.find()` para pegar o produto de `id === 2`.
4. Use `.filter()` para pegar todos os produtos cujo `id` é maior que 1.
5. Use `.findIndex()` para descobrir a posição do produto de `id === 1`.
6. Use `.filter()` para criar um novo array sem o produto de `id === 2`.
7. Percorra o array com `for` e imprima só o `nome` de cada produto.
8. Escreva `existe(id)` que retorna `true` / `false` usando `.find()`.
9. Acesse e imprima o `nome` do primeiro produto do array.
10. Altere o `nome` do produto de índice `0` para `"Webcam"`.

Parte 1 — Complete o código

1. Create

Complete a função que cria um item com id automático.

```
let tarefas = [];  
let proximoId = 1;  
  
function criarTarefa(titulo) {  
  const nova = { id: proximoId, titulo, concluida: false };  
}
```

```
// TODO: adicione 'nova' no array e incremente 'proximoId'  
return nova;  
}  
criarTarefa("Estudar CRUD");  
console.log(tarefas); // [ { id: 1, titulo: 'Estudar CRUD', concluida: false  
  } ]
```

2. Read por ID

Complete a busca.

```
function buscarPorId(id) {  
  // TODO: retorne a tarefa cujo id bate, usando .find()  
}  
console.log(buscarPorId(1)); // { id: 1, titulo: 'Estudar CRUD', ... }
```

3. Update

Complete a atualização usando `.findIndex()`.

```
function atualizarTarefa(id, novoTitulo) {  
  const indice = /* TODO: use .findIndex() */;  
  if (indice === -1) return "Tarefa não encontrada";  
  tarefas[indice].titulo = novoTitulo;  
  return tarefas[indice];  
}
```

Parte 2 — Ache o bug

4. Delete que não deleta

Deveria remover a tarefa, mas o array continua igual. Encontre o erro.

```
function deletarTarefa(id) {  
  tarefas.filter((t) => t.id !== id); // BUG  
  return "Removida";  
}  
// Dica: .filter() retorna um NOVO array. O que fazer com esse retorno?
```

5. Comparação errada

Este `find` nunca acha o item, mesmo passando um id que existe. Por quê?

```
const item = tarefas.find((t) => t.id === id); // BUG sutil
```

Parte 3 — Prever a saída

6. O que sai?

Sem rodar, diga a saída.

```
let lista = [{ id: 1, ok: false }, { id: 2, ok: true }];

console.log(lista.find((x) => x.ok === true)); // (a)
console.log(lista.filter((x) => x.ok === false).length); // (b)
console.log(lista.findIndex((x) => x.id === 2)); // (c)
console.log(lista.find((x) => x.id === 99)); // (d)
```

Parte 4 — Escreva do zero

7. CRUD de Contatos (Desafio)

Implemente um CRUD completo em memória para uma agenda de contatos. Cada contato: `{ id, nome, telefone }`. Escreva as funções:

- `criarContato(nome, telefone)` : cria com id automático (Create / `.push()`).
- `listarContatos()` : retorna todos (Read).
- `buscarPorId(id)` : retorna um (Read / `.find()`).
- `buscarPorNome(termo)` : retorna todos que contêm o termo no nome (Read / `.filter()` + `.includes()`).
- `atualizarContato(id, novosDados)` : atualiza (Update / `.findIndex()`); se não achar, retorna aviso.
- `deletarContato(id)` : remove (Delete / `.filter()`); se não achar, retorna aviso.

Ao final, teste chamando as funções em sequência e imprimindo o array a cada passo.

Dica: o CRUD em array usa quatro métodos. `.push()` para criar. `.find()` para achar um, `.filter()` para achar vários. `.findIndex()` para achar a posição e alterar. E `arr = arr.filter(...)` para apagar, reatribuindo o array sem o item removido.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: CRUD em Arrays - Módulo 06